SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Keerthana.G	**Reg. No.:**	192521446
Date:		**Duration:**	60 minutes

Code of Conduct

I _Keerthana.G (192521446)._____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

AN AI-DRIVEN HOTEL RESERVATION AND ROOM MANAGEMENT PLATFORM WITH DYNAMIC PRICING AND DEMAND FORECASTING

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.

System Objectives

The main objective of the proposed system is to develop an intelligent hotel reservation and room management platform that uses Artificial Intelligence (AI) to automate hotel operations, optimize room pricing, and accurately forecast customer demand.

The objectives of the system are:

- To provide an easy and secure online hotel reservation system.
- To automate room allocation and room management.
- To implement AI-based dynamic pricing based on demand, season, occupancy, and market trends.
- To predict future room demand using demand forecasting techniques.
- To reduce manual intervention in hotel operations.
- To maximize hotel revenue through optimized pricing strategies.
- To provide real-time room availability and booking updates.
- To generate analytical reports for hotel administrators to support decision-making.
- To enhance customer satisfaction through personalized and efficient services.

- Analyze the functional and non-functional requirements that influence the software architecture.

  The system should provide the following functionalities:

  Customer Management

  - User registration and secure login.
  - Profile creation and management.
  - Password recovery and account verification.

  Hotel & Room Management

  - Add, update, delete, and view hotel rooms.
  - Categorize rooms based on type and amenities.
  - Track room availability in real time.
  - Manage room maintenance status.

  Reservation Management

  - Search available rooms.
  - Book rooms online.
  - Modify or cancel reservations.
  - Generate booking confirmation.
  - Maintain booking history.

  AI-Based Dynamic Pricing

  - Analyze occupancy rate.
  - Monitor seasonal demand.
  - Consider local events and holidays.
  - Adjust room prices automatically.
  - Recommend optimal room prices.

Demand Forecasting

- Predict future booking demand.
- Analyze historical reservation data.
- Forecast occupancy levels.
- Generate demand prediction reports.

Payment Management

- Online payment processing.
- Multiple payment options.
- Payment verification.
- Invoice generation.

Notification System

- Booking confirmation notifications.
- Cancellation notifications.
- Payment reminders.
- Promotional offers.

Reporting and Analytics

- Revenue reports.
- Occupancy reports.
- Customer analytics.
- Pricing trend analysis.
- Demand forecasting reports.

Administration

- Manage users.
- Manage hotel information.
- Manage room inventory.
- View dashboards.
- Monitor AI recommendations.

Non-Functional Requirements

Performance

- The system should process booking requests within **2–3 seconds**.
- Real-time room availability should be updated immediately.
- AI pricing recommendations should be generated quickly.

Scalability

- The platform should support thousands of concurrent users.
- It should accommodate multiple hotels and future expansion.

Reliability

- The system should ensure accurate booking information.
- Data loss should be minimized through backup mechanisms.

Availability

- The system should be available **24×7** with minimal downtime.
- Backup servers should ensure continuous operation.

Security

- Secure user authentication and authorization.
- Password encryption.
- Secure payment transactions.
- Data privacy protection.
- Protection against unauthorized access and cyber threats.

Maintainability

- Modular architecture for easy maintenance.
- Easy software updates and bug fixes.

- Well-documented code and APIs.

Usability

- User-friendly interface.
- Responsive web and mobile access.
- Simple navigation and booking process.

Compatibility

- Compatible with major web browsers.
- Supports desktop, tablet, and mobile devices.

Data Integrity

- Accurate storage of reservation and pricing data.
- Prevention of duplicate bookings.

Recoverability

- Automatic database backup.
- Disaster recovery mechanism.
- Quick restoration after failures.

- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

1. Scalability

- Supports a growing number of users and hotels.
- Handles increasing booking requests efficiently.
- Accommodates AI-based pricing and demand forecasting as data grows.
- Maintains system performance during peak seasons.

2. Maintainability

- Uses a modular architecture for easier updates.
- Simplifies bug fixing and system maintenance.
- Supports the addition of new features with minimal changes.
- Allows easy updating of AI models and business rules.

3. Reliability

- Ensures accurate reservation and pricing information.
- Prevents duplicate bookings and data inconsistencies.
- Maintains stable system operation with minimal failures.
- Preserves data integrity through regular backups.

4. Availability

- Provides 24×7 access to the booking platform.
- Minimizes downtime through backup servers and redundancy.
- Supports continuous hotel operations during failures.
- Ensures quick recovery using failover mechanisms.

5. Performance

- Delivers fast room search and booking responses.
- Updates room availability in real time.
- Generates AI pricing recommendations quickly.
- Handles multiple users simultaneously with low response time.

6. Security

- Provides secure user authentication and authorization.
- Encrypts sensitive customer and payment data.
- Protects the system from unauthorized access and cyber threats.
- Implements secure APIs and role-based access control (RBAC).

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC,

Selected Architecture: Hybrid Architecture (Microservices + MVC + Client-Server)

The proposed **AI-Driven Hotel Reservation and Room Management Platform** adopts a **Hybrid Architecture** that combines **Microservices, MVC (Model–View–Controller), and Client–Server Architecture**.

- **Client–Server Architecture** enables communication between users (customers and administrators) and the central server.
- **MVC Architecture** separates the application into Model, View, and Controller, making development and maintenance easier.
- **Microservices Architecture** divides the system into independent services such as Booking, Room Management, Dynamic Pricing, Demand Forecasting, Payment, and Notification, allowing each service to operate and scale independently.

This hybrid approach provides flexibility, scalability, maintainability, and efficient AI integration.

- Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).

1. Supports Independent Modules

- Each major function (Booking, Room Management, Pricing, AI Forecasting, Payment, Notification) is developed as an independent service.
- Changes to one module do not affect others.

2. High Scalability

- Individual services can be scaled independently based on demand.
- Supports thousands of concurrent users and multiple hotels without affecting performance.

3. Easy Maintenance

- MVC separates the user interface, business logic, and data management.
- Modular services simplify debugging, testing, and software updates.

4. Better Performance

- Independent microservices process requests simultaneously.
- Faster booking, room availability updates, and AI-based pricing recommendations.

- Justify why the selected architecture is the most suitable for the given problem.

Reliable and Fault-Tolerant

- Failure in one service (e.g., Notification Service) does not stop the entire system.
- Other services continue operating normally.

Easy AI Integration

- Dynamic Pricing and Demand Forecasting can be implemented as separate AI services.
- AI models can be updated without affecting the reservation system.

Enhanced Security

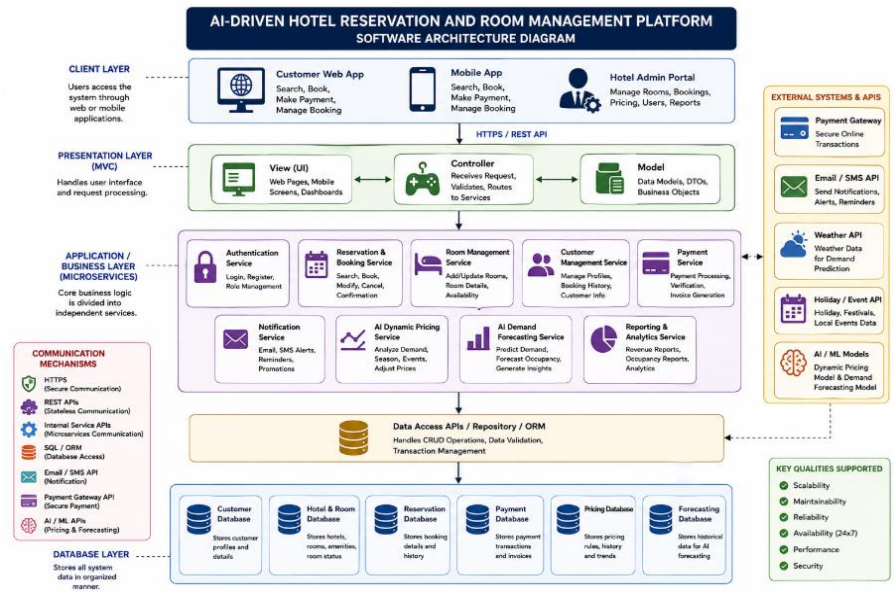
- Authentication, payment, and user management services can implement dedicated security measures.
- Supports secure APIs, encryption, and role-based access control.

Future Expansion

- New features such as chatbot support, loyalty programs, or recommendation systems can be added as new microservices without redesigning the entire architecture.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.



- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.

1. Client Layer

- Customer Web Application
- Mobile Application
- Hotel Admin Portal

2. Presentation Layer (MVC)

- User Interface (View)
- Controller
- Model

3. Application / Business Layer (Microservices)

- Authentication Service

- Customer Management Service
- Reservation & Booking Service
- Room Management Service
- Payment Service
- Notification Service
- AI Dynamic Pricing Service
- AI Demand Forecasting Service
- Reporting & Analytics Service

4. Data Access Layer

- Repository Pattern
- ORM (Object Relational Mapping)
- Database Access APIs

5. Database Layer

- Customer Database
- Hotel Database
- Room Database
- Reservation Database
- Payment Database
- Pricing Database
- Demand Forecasting Database

- Use appropriate software architecture notations and labels.

- Use **rectangles** to represent system layers, components, and services.

- Use **cylinders** to represent databases.
- Use **arrows (→)** to indicate the flow of data and communication.
- Use **bidirectional arrows (↔)** to show two-way communication between services.

- Use **clouds or external boxes** to represent third-party systems and external APIs.
- Label each architectural layer clearly (Client Layer, Presentation Layer, Business Layer, Data Access Layer, Database Layer).
- Label all major components such as Authentication, Reservation, Room Management, Payment, Notification, AI Dynamic Pricing, Demand Forecasting, and Reporting.
- Label all databases, including Customer Database, Hotel Database, Room Database, Reservation Database, Payment Database, Pricing Database, and Forecasting Database.
- Label external systems such as Payment Gateway, Email/SMS API, Weather API, Holiday/Event API, and AI/ML Services.
- Label communication mechanisms such as HTTPS, REST APIs, SQL/ORM, Internal Service APIs, and JSON Data Exchange.
- Maintain consistent symbols, colors, and naming conventions throughout the architecture diagram.
- Arrange components logically to clearly show the interaction between layers and the flow of information.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

Client Layer

Purpose: Provides the interface for customers and hotel administrators to interact with the system.

Responsibilities:

- Display hotel and room information.
- Allow users to search and book rooms.
- Enable administrators to manage hotel operations.
- Send user requests to the server and display responses.

2. Presentation Layer (MVC)

Purpose: Manages user interaction and request processing.

Responsibilities:

- Display user interfaces and dashboards.
- Validate user input.
- Process user requests.
- Forward requests to the business layer.

3. Authentication Service

Purpose: Ensures secure access to the system.

Responsibilities:

- User registration and login.
- Verify user identity.
- Manage user roles and permissions.
- Encrypt passwords and secure authentication.

4. Reservation & Booking Service

Purpose: Handles hotel room reservation operations.

Responsibilities:

- Search available rooms.
- Create, modify, and cancel bookings.
- Update reservation records.
- Generate booking confirmations.

5. Room Management Service

Purpose: Manages hotel room information and availability.

Responsibilities:

- Add, update, and remove room details.
- Track room availability.
- Manage room status and maintenance.
- Update room inventory.

6. Customer Management Service

Purpose: Maintains customer information.

Responsibilities:

- Store customer profiles.
- Maintain booking history.
- Update customer details.
- Manage customer preferences.

7. Payment Service

Purpose: Processes secure online payments.

Responsibilities:

- Process payment transactions.
- Verify payment status.
- Generate invoices.
- Handle refunds and payment records.

8. Notification Service

Purpose: Sends notifications to users.

Responsibilities:

- Send booking confirmations.
- Deliver payment reminders.
- Notify booking cancellations.
- Send promotional messages.

9. AI Dynamic Pricing Service

Purpose: Optimizes hotel room pricing using AI.

Responsibilities:

- Analyze occupancy rates.
- Monitor seasonal demand.
- Evaluate holidays and local events.
- Automatically adjust room prices.

10. AI Demand Forecasting Service

Purpose: Predicts future booking demand.

Responsibilities:

- Analyze historical booking data.

- Forecast occupancy levels.
- Predict customer demand.
- Support pricing decisions.

11. Reporting & Analytics Service

Purpose: Generates business insights and reports.

Responsibilities:

- Generate revenue reports.
- Analyze occupancy trends.
- Produce booking statistics.
- Display AI forecasting reports.

12. Data Access Layer

Purpose: Connects the application with the database.

Responsibilities:

- Manage database connections.
- Execute database queries.
- Retrieve and store data.
- Maintain

- Explain how the components communicate and exchange data.

his layer contains the core business logic of the system and consists of several independent microservices.

Authentication Service

- User registration and login.
- Role-based access control.
- Password encryption and authentication.

Reservation & Booking Service

- Room search and availability checking.
- Booking creation, modification, and cancellation.
- Booking confirmation generation.

Room Management Service

- Add, edit, and remove rooms.
- Update room status.
- Track room availability.

Customer Management Service

- Manage customer profiles.
- Store booking history.
- Maintain customer information.

Payment Service

- Process online payments.
- Verify transactions.
- Generate invoices.

Notification Service

- Send booking confirmations.
- Payment reminders.
- Cancellation alerts.
- Promotional notifications.

AI Dynamic Pricing Service

- Analyze occupancy rates.
- Monitor seasonal demand.
- Evaluate holidays and local events.

- Automatically adjust room prices.

AI Demand Forecasting Service

- Analyze historical booking data.
- Predict future occupancy.
- Forecast customer demand.
- Support pricing decisions.

Reporting & Analytics Service

- Revenue reports.
- Occupancy reports.
- Booking statistics.
- Customer analytics.
- AI prediction reports.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability

    - Supports a large number of users, hotels, and booking requests.
    - Allows individual microservices to scale independently based on demand.
    - Handles increased AI processing for dynamic pricing and demand forecasting.
    - Supports future expansion by adding new services without affecting the existing system.

- Security

- Implements secure user authentication and authorization.
- Encrypts sensitive customer and payment information.
- Uses HTTPS for secure communication between clients and the server.
- Integrates secure payment gateway APIs for online transactions.
- Applies role-based access control (RBAC) to protect system resources.

- Performance

- Provides fast room search and booking responses.
- Updates room availability in real time.
- Generates AI-based pricing recommendations efficiently.
- Supports multiple concurrent users with minimal response time.
- Optimizes database access using efficient queries and caching.

- Reliability

- Ensures accurate booking and pricing information.
- Prevents duplicate bookings and maintains data consistency.
- Performs regular database backups to prevent data loss.
- Isolates failures so one service does not affect the entire system.
- Maintains consistent system operation with minimal errors.

- Maintainability

- Uses a modular hybrid architecture for easier maintenance.
- Separates presentation, business logic, and data access layers.
- Allows independent updates and testing of microservices.
- Simplifies debugging, feature enhancement, and AI model updates.
- Supports code reusability and easier future development.

- Availability

  - Provides 24×7 access to hotel booking and management services.
  - Uses backup servers and redundancy to minimize downtime.
  - Supports failover mechanisms for continuous operation.
  - Ensures uninterrupted access during maintenance or server failures.
  - Maintains consistent service availability for customers and administrators.

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.

  The proposed system adopts a **Hybrid Architecture (Client–Server + MVC + Microservices)** because it combines the strengths of multiple architectural styles to meet the requirements of an AI-driven hotel reservation platform.

  - **Client–Server Architecture** enables secure communication between users and the central server.
  - **MVC Architecture** separates the user interface, business logic, and data, making the application easier to develop and maintain.
  - **Microservices Architecture** divides the system into independent services such as Reservation, Room Management, Payment, AI Dynamic Pricing, and Demand Forecasting, allowing each service to operate and scale independently.

- The architecture supports AI integration while ensuring flexibility, reliability, and high performance.

- Discuss the trade-offs considered while selecting the architecture.

  - **Advantages**
    - Provides high scalability by allowing individual services to scale independently.
    - Improves maintainability through modular design.
    - Enhances reliability by isolating failures to individual services.
    - Enables independent deployment and updates without affecting the entire system.
    - Supports seamless integration of AI-based pricing and demand forecasting.
  - **Trade-offs**
    - More complex to design and manage than a simple layered architecture.
    - Requires additional communication between microservices through APIs.
    - Deployment and monitoring require more infrastructure and configuration.
    - Initial development cost and setup time are higher due to multiple services.

Despite these trade-offs, the benefits of flexibility, scalability, and maintainability make the hybrid architecture the most suitable choice for this platform.

- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

  Support for Future Enhancements, System Integration, and Long-Term Maintainability

  Future Enhancements

- New features such as AI chatbots, loyalty programs, recommendation systems, multilingual support, and smart room management can be added as separate microservices.
- Existing services can be upgraded independently without affecting other modules.

System Integration

- REST APIs enable easy integration with payment gateways, email/SMS services, travel booking platforms, weather services, and third-party hotel management systems.
- AI models for dynamic pricing and demand forecasting can be updated or replaced without changing the core application.

Long-Term Maintainability

- The modular architecture simplifies testing, debugging, and maintenance.
- Clear separation of concerns (presentation, business logic, and data access) improves code organization.
- Independent microservices allow easier updates, bug fixes, and feature enhancements.
- Standard APIs and reusable components ensure the system remains adaptable to future business and technological changes.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.

	clearly explains quality attributes influencing the architecture.			
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification	Provides strong justification for	Provides adequate justification with	Limited justification	Justification is weak or absent;

and Technical Presentation (10%)	architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	minor gaps; report is well organized.	with minimal discussion of trade-offs or future scope; report needs improvement.	report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25